SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Athavan K	**Reg. No.:**	192524036
Section / Batch:	2nd year	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Athavan K

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				

Total Marks (100):	

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim:

To write a C program to find the length of the Longest Increasing Subsequence (LIS) of a given sequence of integers using Dynamic Programming, such that the chosen elements appear in strictly increasing order while preserving their original relative order in the sequence.

Problem Statement

Given a sequence of integers, determine the length of the longest subsequence in which the elements appear in strictly increasing order. The subsequence must preserve the original order of elements, but the chosen elements need not be adjacent. The goal is to identify the maximum possible increasing subsequence length.

Input Format:

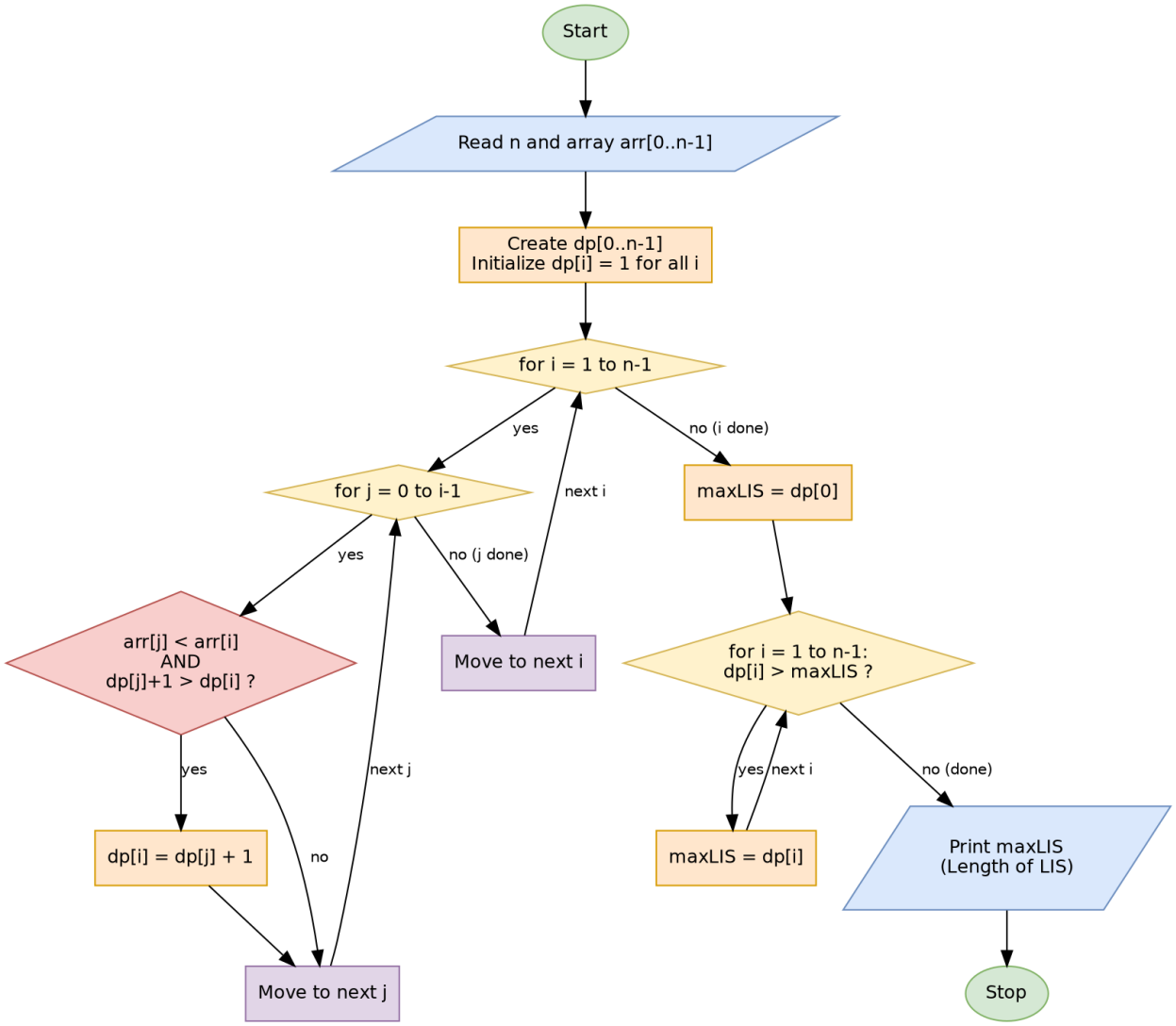
```
n      = number of elements
arr[]  = sequence elements
```

Algorithm

1. Start.
2. Read the number of elements n and the array arr[0..n-1].
3. Create an auxiliary array dp[0..n-1], where dp[i] stores the length of the longest increasing subsequence ending at index i.
4. Initialize dp[i] = 1 for every i, since every single element is an increasing subsequence of length 1 by itself.
5. For each index i from 1 to n-1, repeat the following:
6. For each index j from 0 to i-1, check if arr[j] < arr[i] AND dp[j] + 1 > dp[i]. If true, update dp[i] = dp[j] + 1.
7. After filling the dp array, find maxLIS as the maximum value among all dp[i] for i = 0 to n-1.
8. Print maxLIS as the length of the Longest Increasing Subsequence.
9. Stop.

Time Complexity: $O(n^2)$ Space Complexity: O(n)

Flowchart


Start
Read n and array arr[0..n-1]
Create dp[0..n-1]
Initialize dp[i] = 1 for all i
for i = 1 to n-1
yes
no (i done)
for j = 0 to i-1
next i
maxLIS = dp[0]
yes
no (j done)
arr[j] < arr[i]
AND
dp[j]+1 > dp[i] ?
Move to next i
for i = 1 to n-1:
dp[i] > maxLIS ?
yes
next j
yes
next i
no (done)
dp[i] = dp[j] + 1
no
maxLIS = dp[i]
Print maxLIS
(Length of LIS)
Move to next j
Stop

Source Code (C Program)

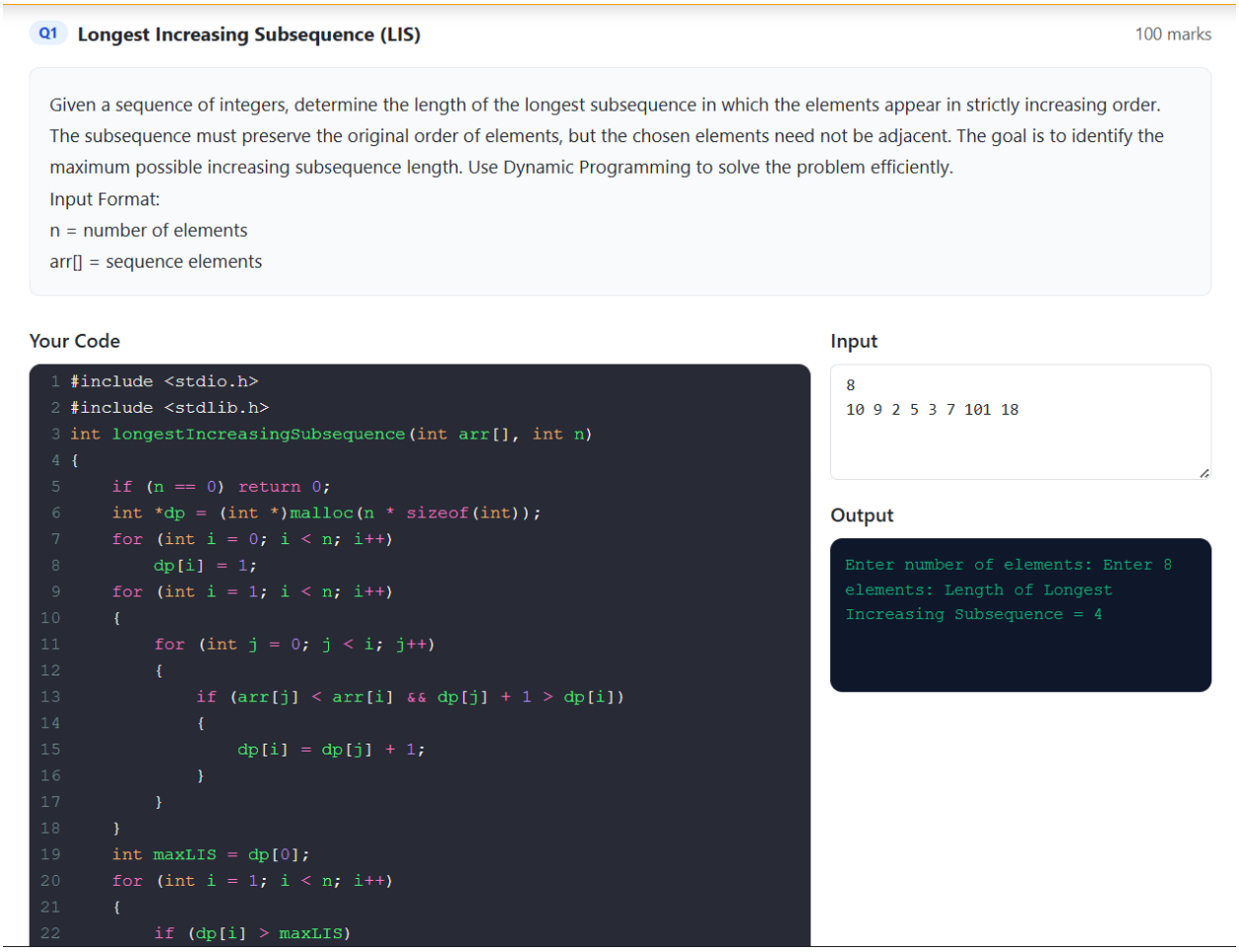
```
#include <stdio.h>
#include <stdlib.h>

int longestIncreasingSubsequence(int arr[], int n)
{
    if (n == 0) return 0;
    int *dp = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++)
        dp[i] = 1;
    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (arr[j] < arr[i] && dp[j] + 1 > dp[i])
            {
                dp[i] = dp[j] + 1;
            }
        }
    }
    int maxLIS = dp[0];
    for (int i = 1; i < n; i++)
    {
        if (dp[i] > maxLIS)
            maxLIS = dp[i];
    }
    free(dp);
    return maxLIS;
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int *arr = (int *)malloc(n * sizeof(int));
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &arr[i]);
    int result = longestIncreasingSubsequence(arr, n);
    printf("Length of Longest Increasing Subsequence =
%d\n", result);
    free(arr);
    return 0;
}
```

Output / Execution Screenshot

The program was compiled and executed successfully. For the sample input n = 8 and arr[] = {10, 9, 2, 5, 3, 7, 101, 18}, the Longest Increasing Subsequence length obtained is 4 (e.g., the subsequence 2, 5, 7, 18 — or 2, 5, 7, 101). The screenshot of the submitted code along with the input/output is shown below.



Q1 **Longest Increasing Subsequence (LIS)** 100 marks

Given a sequence of integers, determine the length of the longest subsequence in which the elements appear in strictly increasing order. The subsequence must preserve the original order of elements, but the chosen elements need not be adjacent. The goal is to identify the maximum possible increasing subsequence length. Use Dynamic Programming to solve the problem efficiently.

Input Format:

n = number of elements

arr[] = sequence elements

Your Code

```c
#include <stdio.h>
#include <stdlib.h>
int longestIncreasingSubsequence(int arr[], int n)
{
    if (n == 0) return 0;
    int *dp = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++)
        dp[i] = 1;
    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (arr[j] < arr[i] && dp[j] + 1 > dp[i])
            {
                dp[i] = dp[j] + 1;
            }
        }
    }
    int maxLIS = dp[0];
    for (int i = 1; i < n; i++)
    {
        if (dp[i] > maxLIS)
```

Input

```
8
10 9 2 5 3 7 101 18
```

Output

```
Enter number of elements: Enter 8
elements: Length of Longest
Increasing Subsequence = 4
```

Result

Thus, a C program to find the length of the Longest Increasing Subsequence of a given sequence using Dynamic Programming was written, executed, and verified successfully.